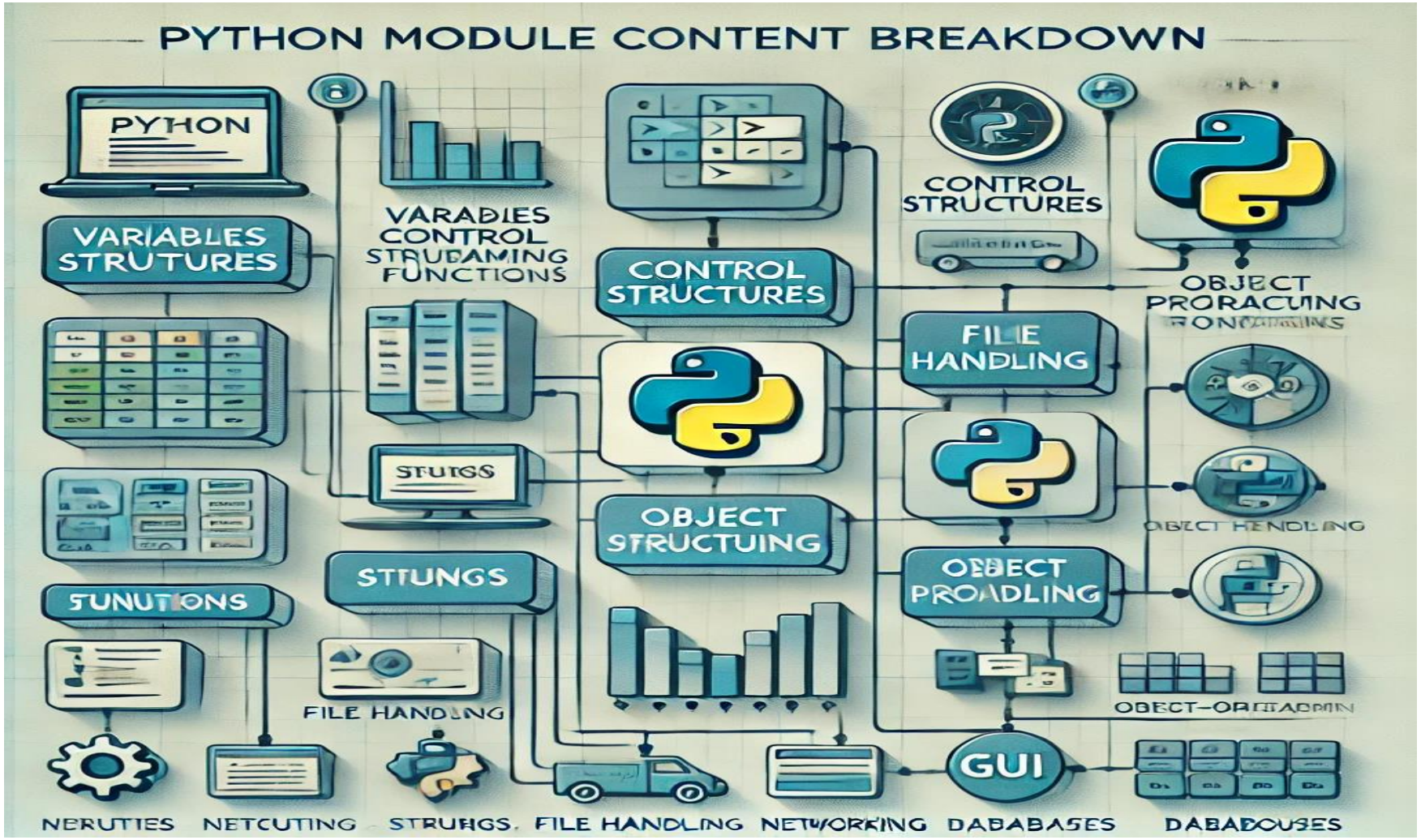
CST1510 Module Introduction



Module structure

- **Course Duration:** 12 Weeks
  Assessment: 100% Coursework (No Exams!)
- **Schedule Overview:**
- **Teaching Sessions (30%):**
  - **2-hour Workshop:** Interactive learning to master core concepts.
  - **3-hour Lab:** Hands-on coding with real-world tasks and challenges.
  - **1-hour Drop-in Session:** Get personalized help and answers to your questions.
- **Independent Study (70%):**
- **Explore Python Documentation**: Dive deep into official resources to enhance your understanding
- **Practice Makes Perfect**: The more you code, the better you'll get!

Module syllabus

Week	Topics Covered	What You'll Learn
Week 1: Getting Started with Python	Variables, simple data types (int, float, string), basic I/O operations (input, print), and introductory Python syntax.	Get comfortable with the basics of Python programming and understand the fundamental building blocks of Python code.
Week 2: Control Structures and Lists	Conditional statements (if, elif, else), loops (for, while), basic list operations, and list comprehensions.	Control the flow of programs, handle lists, and write efficient decision-making code.
Week 3: Functions, Lambda Expressions, and Multidimensional Lists	Defining and using functions, lambda expressions for concise coding, and working with multidimensional lists (nested lists).	Create reusable code with functions and explore advanced data structures.
Week 4: Strings, Special Methods, Modules, and Regular Expressions	Advanced string manipulation, special methods, using modules and packages, and an introduction to regular expressions.	Process and manipulate text data, and leverage existing code to build complex applications.
Week 5: File Handling, Exceptions, and Data Structures including Pandas	File I/O (reading, writing, and parsing), handling exceptions, tuples, sets, dictionaries, and an introduction to Pandas for data manipulation.	Work with files, manage data robustly, and manipulate large datasets with Pandas.
Week 6: Portfolio Submission and Project Work	Portfolio compilation of work from Weeks 1-5, covering variables, control structures, functions, file handling, exception management, and Pandas. Start on a comprehensive project integrating all learned skills.	Showcase your understanding through a portfolio of key assignments, reflecting on your progress so far, and begin a practical project that ties together all concepts from the first half of the course.
Week 7: Object-Oriented Programming, Inheritance, and Polymorphism	Core OOP principles (classes, objects), inheritance, polymorphism, and applying these concepts in real-world scenarios.	Dive into object-oriented programming to write complex, organized, and reusable code. Learn how to model real-world problems using classes and objects.
Week 8: Networking and Data Analysis	Advanced networking concepts using Python (socket programming, protocols like HTTP/HTTPS) and data analysis techniques focused on networking data.	Integrate networking skills with data analysis, learning to build simple networked applications and analyze data.
Week 9: SQL and NoSQL Databases with Python	Working with SQL databases (sqlite3, SQLAlchemy) and NoSQL databases (e.g., MongoDB) and their integration with Python.	Master the skills needed to interact with both relational and non-relational databases. Learn how to store, retrieve, and manipulate data efficiently in Python.
Week 10: GUI Development with Tkinter	Building graphical user interfaces (GUIs) using Tkinter, including layout management, widgets, event handling, and creating interactive applications.	You'll learn how to create user-friendly interfaces for your Python applications using Tkinter.
Week 11: Web Scraping and Working with URLs and Requests	Web scraping with libraries like BeautifulSoup and Scrapy, and handling HTTP requests with the requests library.	You'll learn how to extract and process information from websites
Week 12: Final Project	A comprehensive project that brings together all the skills and knowledge you've gained throughout the course.	You'll tackle a real-world problem involving programming, data communications, and networking, showcasing your ability to apply what you've learned in a practical, integrative way.

Assessments:

Formative Assessments Schedule

Formative assessment	Deadline
Week 1 – 6 SOBs	Until week 6

Summative Assessments Schedule

Summative assessment	Weighting	Deadline	Feedback
Portfolio of Practical tasks	50%	Week 6	15 days after submission
1000-word report + code and testing	50%	Week 12	15 days after submission

Suggestions for successful study:

- **Be Present & Engaged:** Attend every session—showing up is half the battle!
- **Stay Focused:** Make the most of your 2-hour workshop by actively paying attention, and fully immerse yourself in the 3-hour lab activities.
- **Plan Your Study Time:** Treat your independent study just like your workshops—schedule it consistently and stick to it.
- **Eliminate Distractions:** During study time, focus on one task or topic at a time. Turn off distractions to maximize your learning.
- **Take Breaks to Recharge:** After a solid study session, give yourself a break—go for a walk, clear your mind, and refresh.
- **Teach What You Learn:** One of the best ways to understand a concept is to explain it to someone else. Share your knowledge!

Any Questions about the module?

Python

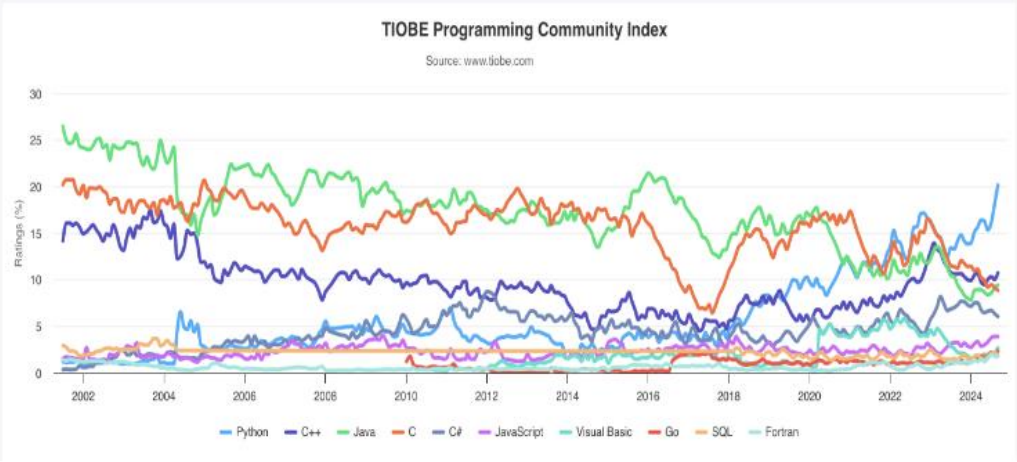
- Python is an object-oriented scripting language
- Released publicly in 1991
- Developed by Guido van Rossum of the National Research Institute for Mathematics and Computer Science in Amsterdam.
- Has rapidly become one of the world’s most popular programming languages
- Particularly popular for educational and scientific computing

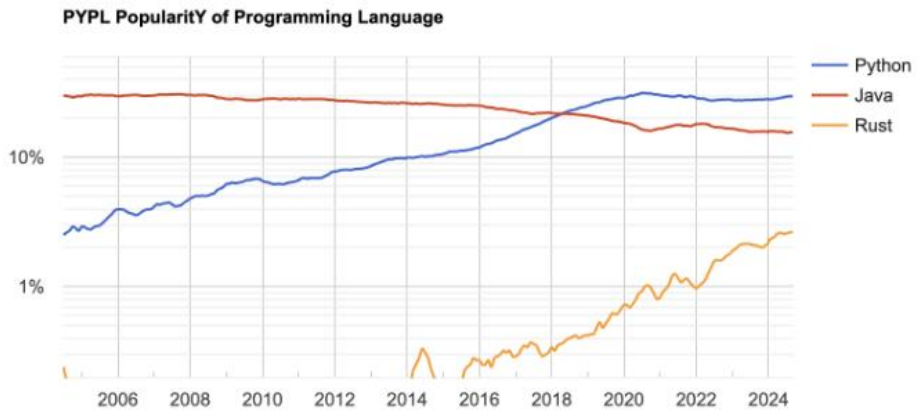
Why Python?

- **Open source**, freely accessible, and supported by a large, active community of contributors.
- **Simpler to learn** than many other programming languages, allowing beginners and experienced developers to become proficient quickly.
- Enables programmers **accomplish complex tasks with less code**.
- Abundant free, open-source Python applications available.
- Widely utilized in web development frameworks like Django and Flask.
- Supports multiple programming paradigms, including procedural, functional, and object-oriented programming.
- Can build anything from basic scripts to complex applications with millions of users, such as Dropbox, YouTube, Reddit, Instagram, and Quora.
- **Highly popular in AI**, which is rapidly growing partly due to Python's strong connection with data science.
- Frequently **used in the financial sector**.
- Offers extensive job opportunities for Python developers across various fields, especially in **data science**, with Python roles being among the highest-paying programming jobs.

Why Python ?

- TIOBE is an index indicating the **popularity of programming languages**. Let's check where it Python is as now: https://www.tiobe.com/tiobe-index/
- The PYPL is another index generating an index based on **how often the language is searched on Google**. https://pypl.github.io/PYPL.html It's a baseline skill for; Data science, data analysis, AI, cybersecurity, DB and network automation, web development and more.





Installing Python

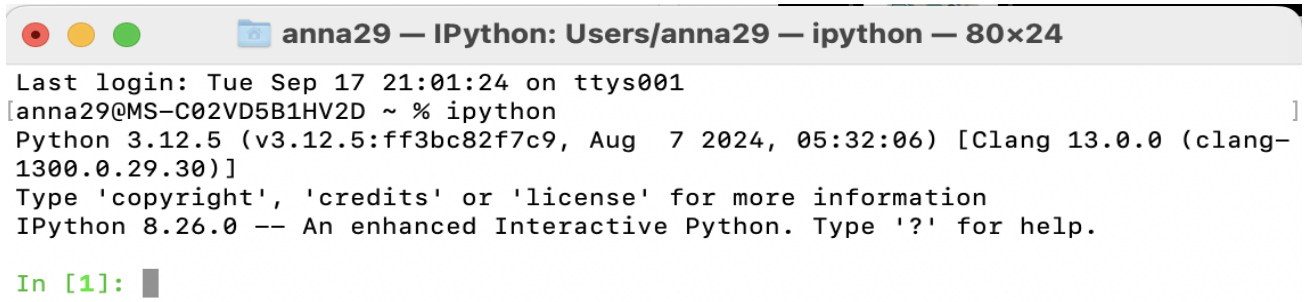
- In the CST1510 module, we will be using Python 3 (as of September 2024, the current version is Python 3.12.6).
- For your home setup, please follow the instructions in the *Python Installation and Setup* guide which also includes information about Python's location, setting the PATH, and more.
- University computers use the *Apps-Anywhere* system, which allows you to install Python as needed. However, please note that all data, including the Python installation, will be erased once you turn off the computer.

Using IPython, IDLE, Jupyter Notebooks and other Development Environments

- Test-drive the IPython interpreter in two modes:
- **interactive mode**—input small pieces of code, known as **snippets**, and instantly view the results
- **script mode**—run code from a file with the .py extension (which stands for Python)
  - These files are referred to as **scripts** or **programs**

I. Interactive Mode

- Open a command-line window on your system
  - On macOS, open a **Terminal** from the **Applications** folder's **Utilities** subfolder
  - On Windows, open the **Command Prompt** from the start menu (type CMD and hit *Enter*)
  - On Linux, open your system's **Terminal** or shell (this varies by Linux distribution)
- Type ipython, then press *Enter* (or *Return*)
- You'll see text like:



Interactive Mode

anna29 — IPython: Users/anna29 — ipython — 80×24

```
Last login: Tue Sep 17 21:01:24 on ttys001
anna29@MS-C02VD5B1HV2D ~ % ipython
Python 3.12.5 (v3.12.5:ff3bc82f7c9, Aug  7 2024, 05:32:06) [Clang 13.0.0 (clang-
1300.0.29.30)]
Type 'copyright', 'credits' or 'license' for more information
IPython 8.26.0 -- An enhanced Interactive Python. Type '?' for help.

In [1]: 12 + 3
Out[1]: 15

In [2]: 5 * (12.7 - 4) / 2
Out[2]: 21.75

In [3]:
```

Interactive Mode

```
anna29 — IPython: Users/anna29 — ipython — 80×24
Last login: Tue Sep 17 21:01:24 on ttys001
anna29@MS-C02VD5B1HV2D ~ % ipython
Python 3.12.5 (v3.12.5:ff3bc82f7c9, Aug  7 2024, 05:32:06) [Clang 13.0.0 (clang-
1300.0.29.30)]
Type 'copyright', 'credits' or 'license' for more information
IPython 8.26.0 -- An enhanced Interactive Python. Type '?' for help.

In [1]: 12 + 3
Out[1]: 15

In [2]: 5 * (12.7 - 4) / 2
Out[2]: 21.75

In [3]:
```

- To leave interactive mode:
  - Type exit and press *Enter* to exit immediately
  - Type *Ctrl + d* (or *control + d*) then confirm
  - Type *Ctrl + d* (or *control + d*) twice

Online editors are

- **jupyter notebook** https://jupyter.org/
- **online-python** https://www.online-python.com/
- **programiz** https://www.programiz.com/python-programming/online-compiler/
- **replit** https://replit.com/languages/python3
- **w3schools** https://www.w3schools.com/python/trypython.asp?filename=demo_compiler
- **trinket** https://trinket.io/embed/python3/a5bd54189b
- **onlinegdb** https://www.onlinegdb.com/online_python_compiler
- **jdoodle** https://www.jdoodle.com/python3-programming-online/

II. Script mode

- Usually created in a text editor that allows you to type and edit code.
- **Integrated Development Environments (IDEs)** offer tools that facilitate the complete software development lifecycle.
  - These include editors, debuggers for finding and fixing logic errors, and more.
- Common Python IDEs are:
  - **PyCharm** https://www.jetbrains.com/pycharm/download/?section=windows#section=windows
  - **Visualstudio** https://visualstudio.microsoft.com/vs/features/python/
  - **Sublime Text** https://www.sublimetext.com/
  - **Atom** https://atom.io/

IDLE

- Python comes with an Integrated Development Environment (IDE) called IDLE
- Open a Python **IDLE** on your system:
  - On **macOS**, open a Terminal from the Applications folder's Utilities subfolder and type IDLE3, then press Enter.
  - On **Windows**, open the Command Prompt from the Start menu (type CMD and hit Enter), then type IDLE and press Enter.
-

IDLE

IDLE SHELL:

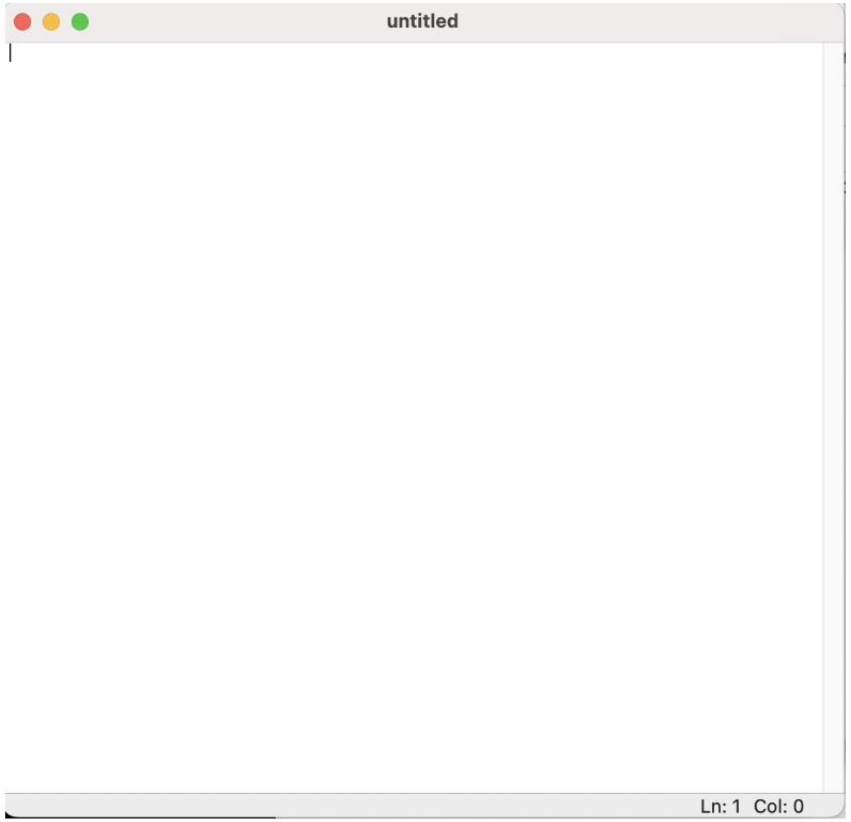
Interactive mode

Powerful interactive calculator

IDLE EDITOR:

Allows creating and saving Python source code files

Files should be saved with the extension .py (filename.py)



You are free to use whichever code editor you prefer for your work in this module

> *—whether it's IDLE, Jupyter Notebook, VS Code, PyCharm, or something else that suits your style.*

Choose what makes you feel most comfortable, and enjoy the process of exploring and learning Python throughout your studies!

Using AI Tools in Your Python Studies

- **Tips for Effective Use**
  - **Don’t rely too much**: AI tools should support your learning, not replace it.
  - **Understand the “why” behind the code**: Always take time to learn the logic and syntax, even if AI suggests solutions.
  - **Experiment and explore**: Use AI to guide your experimentation but ensure you build problem-solving skills yourself.
- **Ethical Considerations**
  - **Use AI responsibly**: Make sure that AI-generated code is a tool for learning, not just a shortcut for assignments.
  - **Academic Integrity**: Be transparent about using AI tools in your work.